

Task 3.1

The Registration Policy:

attach(): if the pointer is null, do nothing - never store null
if the pointer is already in the list, do nothing - never store duplicates

detach(): if the pointer is not in the list, do nothing - nothing to delete
only forgets the address, the observer is never deleted

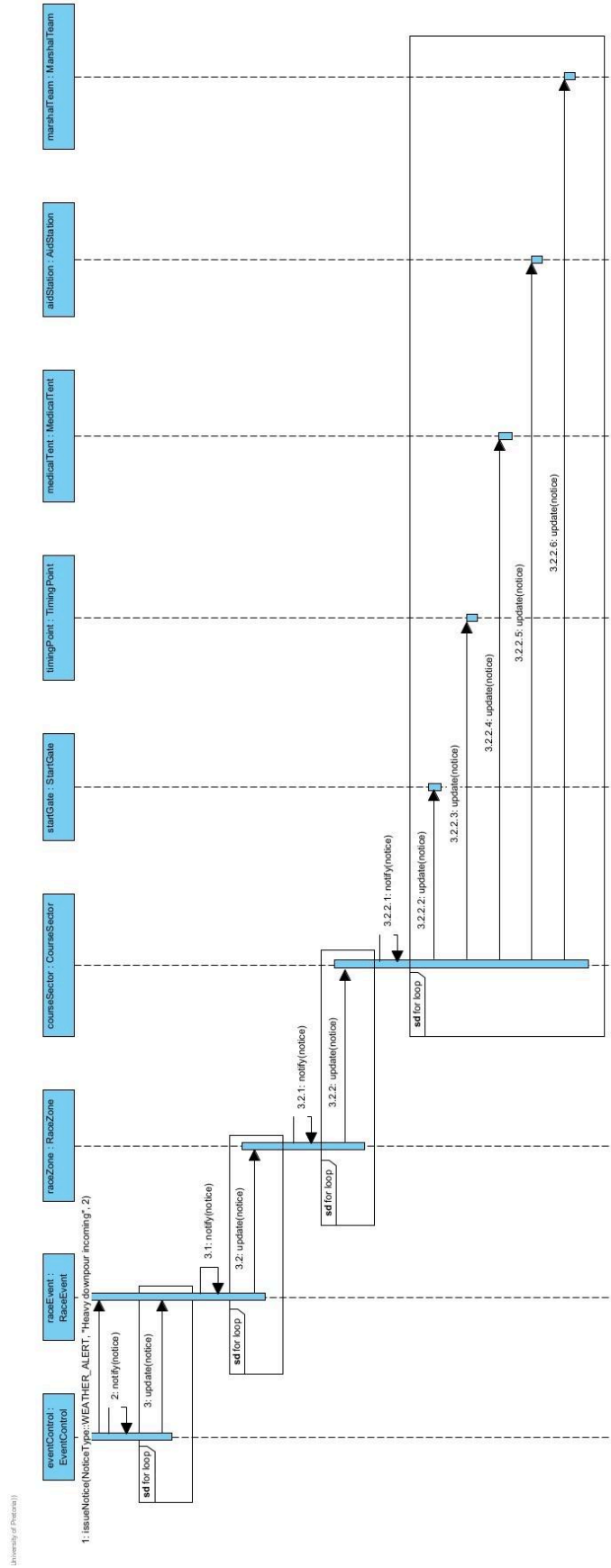
Task 3.2

This design guarantees lifetime ordering in a defensible way, as it follows a guaranteed destruction order. The ownership hierarchy ensures that all observers are destroyed before their subjects

It doesn't include observers detaching themselves, as observers do not know of their subjects, therefore they have no way of knowing which subjects hold their pointers, and cannot self-detach.

It also doesn't include subjects cleaning registers, as subjects are unaware of the lifetime of their observers, and therefore cannot clean up after their detachment.

Task 3.4



Task 3.5

Push was chosen for the Observer implementation. This involves the subject taking responsibility by packaging its public state and providing observers with this when issuing the update function. In turn, the sequence diagrams are now explicit.

void update(const Notice& notice);

- When a subject notifies its observers, it pushes a complete Notice object containing all relevant information

```
class Notice {  
    NoticeType type; //e.g. WEATHER_ALERT  
    std::string message; //description  
    int severity; //e.g. 1 = low, etc.  
};
```

Every observer can then decide to use the immutable Notice without keeping a Subject pointer or making additional calls.

The tradeoff is low flexibility, as now all observers depend on the Notice representation and may receive fields they do not use. This could increase coupling between hierarchies, but is acceptable here because Notice is deliberately small and stable.